FASE 1 · PROYECTO 02

procs

Procesos, pipes y señales:
las piezas con las que se construye una shell

Versión 1.0 · Roadmap Sistemas Linux

Resumen

Cinco ejercicios para dominar el ciclo de vida de un proceso (`fork`, `execve`, `wait`), la comunicación mediante pipes, el manejo correcto de señales y los grupos de procesos. Todo lo que aprendas aquí lo vas a necesitar, sin excepción, en `psh`.

CAPÍTULO I

Introducción

En Unix, crear un proceso y ejecutar un programa son dos operaciones separadas. `fork` duplica el proceso actual (memoria, file descriptors, máscara de señales…) y `execve` sustituye la imagen del proceso por un programa nuevo, **conservando** los file descriptors abiertos. Ese hueco entre las dos llamadas es donde una shell configura redirecciones, pipes, grupos de procesos y señales del hijo.

Las señales son la otra mitad del puzzle: notificaciones asíncronas que pueden interrumpir tu programa en cualquier instrucción. Manejarlas bien exige entender qué es una función *async-signal-safe* y por qué la mayoría de la libc no lo es.

CAPÍTULO II

Instrucciones generales

- C, mismas flags que en `rawio`. Puedes usar `stdio` en este proyecto salvo **dentro de los handlers de señales**.
- Funciones permitidas: todas las syscalls de procesos, pipes y señales (`fork`, `execve`, `waitpid`, `wait4`, `pipe`, `dup2`, `kill`, `sigaction`, `sigprocmask`, `setpgid`, `tcsetpgrp`…), más `malloc`, `free`, `strerror`, `strsignal` y la familia `printf`.
- **Prohibidas**: `system`, `popen`, `execvp`, `execlp` y cualquier otra variante de `exec` que busque en el `PATH` por ti. Tú resolverás el `PATH`.
- Ningún proceso zombi debe sobrevivir a tu programa, y ningún hijo debe heredar file descriptors que no necesita.

CAPÍTULO III

Ejercicios

Ejercicio	00: family
Entrega	`ex00/` · binario `family`, `NOTES.md`
Permitido	`fork`, `waitpid`, `getpid`, `getppid`, `sleep`, `printf`

`./family N` crea N hijos. El hijo *i* imprime su PID y el de su padre, duerme *i* segundos y termina con código de salida *i*. El padre imprime cada terminación **en el orden en que ocurre**, con el PID y el código. En `NOTES.md`, responde con evidencias (salida de `ps`, contenido de `/proc/PID/status`):

1. Si el padre no llama a `wait`, ¿en qué estado quedan los hijos? Muestra la línea `State:` de `/proc`.
2. Si el padre muere antes que los hijos, ¿quién pasa a ser su padre? ¿Qué es un *subreaper* (`prctl(PR_SET_CHILD_SUBREAPER)`)?
3. Haz `printf("hola")` **sin** `\n` justo antes del `fork`. ¿Cuántas veces aparece «hola»? ¿Por qué?

Ejercicio	01: run
Entrega	`ex01/` · binario `run`
Permitido	lo anterior + `execve`, `wait4`, `access`, `getenv`, `clock_gettime`, `strsignal`

`./run cmd [args...]` ejecuta un comando y explica cómo terminó:

```
$ ./run ls /noexiste
ls: cannot access '/noexiste': No such file or directory
[run] exited with status 2 · 0.002s real · 1.0 MiB max RSS
$ ./run sh -c 'kill -SEGV $$'
[run] killed by signal 11 (Segmentation fault) · 0.001s real
$ ./run comando_inventado
run: comando_inventado: command not found
$ echo $?
127
```

- Busca el ejecutable en cada directorio del `PATH` tú mismo. Si el nombre contiene `/`, no se busca.
- Códigos de salida como `bash`: 127 si no se encuentra, 126 si existe pero no es ejecutable, y el código del hijo en otro caso. Si el hijo murió por una señal, devuelve 128 + número de señal.
- Decodifica el estado con `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`, `WTERMSIG` y `WCOREDUMP`.
- Usa `wait4` para obtener la `struct rusage` del hijo (memoria máxima, tiempo de CPU).

Ejercicio	02: pipeline
Entrega	`ex02/` · binario `pipeline`
Permitido	lo anterior + `pipe`, `dup2`, `close`

`./pipeline "cmd1 args" "cmd2 args" ... "cmdN args"` ejecuta N comandos conectados por pipes, equivalente a `cmd1 | cmd2 | ... | cmdN`. Cada argumento se divide por espacios (no hace falta tratar comillas).

- **Todos** los procesos se lanzan antes de esperar a ninguno. Si esperas a `cmd1` antes de lanzar `cmd2`, un `cmd1` que escriba más de 64 KiB se quedará bloqueado para siempre. Demuéstralo en `NOTES.md`.
- Cada proceso debe cerrar todos los extremos de pipe que no use. Si te sobra uno abierto, `./pipeline "yes" "head -n 1"` no terminará nunca. Explica por qué.
- El código de salida del programa es el del último comando, como en una shell.
- N puede ser 1 o 500. No asumas un límite fijo.

```
$ ./pipeline "ls -l /proc/self/fd"
# Solo deben aparecer 0, 1, 2 y el fd del propio directorio: ningún extremo de pipe.
$ ./pipeline "cat /etc/passwd" "cut -d: -f7" "sort" "uniq -c" "sort -rn"
```

Ejercicio	03: sigtour
Entrega	`ex03/` · binario `sigtour`, `NOTES.md`
Permitido	lo anterior + `sigaction`, `sigprocmask`, `sigsuspend`, `poll`, `alarm`

Un programa que se mantiene vivo y reacciona a señales de forma **correcta**:

- `SIGINT` (Ctrl+C): imprime «interrumpido» y sigue funcionando. El tercer `SIGINT` en menos de 2 segundos lo termina.
- `SIGUSR1`: imprime cuántas señales de cada tipo ha recibido.
- `SIGTERM`: termina limpiamente, liberando todo.
- `SIGCHLD`: cada 5 segundos el programa lanza un hijo que hace `sleep 1`; los hijos deben recogerse sin dejar zombis.
- Los handlers solo pueden tocar variables `volatile sig_atomic_t` o escribir en un pipe (*self-pipe trick*). Toda la lógica real (imprimir, contar, recoger hijos) ocurre en el bucle principal, que espera con `poll`.

En `NOTES.md`:

1. Escribe un ejemplo mínimo en el que llamar a `printf` o `malloc` desde un handler provoca un deadlock y explica el mecanismo.
2. ¿Qué cambia el flag `SA_RESTART` en un `read` bloqueado sobre la terminal? Demuéstralo con dos ejecuciones.
3. ¿Por qué puedes recibir un solo `SIGCHLD` aunque hayan terminado tres hijos? ¿Cómo debe ser el bucle de `waitpid`?
4. ¿Qué es una *race condition* entre comprobar una variable y llamar a `pause()`, y cómo la resuelve `sigsuspend`?

Ejercicio	04: pgdemo
Entrega	`ex04/` · binario `pgdemo`, `NOTES.md`
Permitido	lo anterior + `setpgid`, `getpgrp`, `tcgetpgrp`, `tcsetpgrp`, `isatty`

La antesala del job control. `./pgdemo cmd [args]` debe:

1. Lanzar `cmd` en un **grupo de procesos nuevo** y darle el control de la terminal (*foreground*).
2. Mientras corre, Ctrl+C y Ctrl+Z afectan solo a `cmd`, nunca a `pgdemo`.
3. Si `cmd` se detiene (Ctrl+Z), `pgdemo` recupera la terminal, lo indica y, tras pulsar Enter, le devuelve la terminal y lo reanuda con `SIGCONT`.
4. Cuando `cmd` termina, `pgdemo` recupera la terminal y sale con el código de `cmd`.

PISTA

Tanto el padre como el hijo deben llamar a `setpgid` para evitar una carrera: no sabes cuál de los dos se ejecuta primero tras el `fork`. Y el padre necesita ignorar `SIGTTOU` para poder llamar a `tcsetpgrp` desde el background. Lee `man 3 tcsetpgrp` y el capítulo de job control de APUE o TLPI.

Prueba con `./pgdemo vim` y `./pgdemo cat`. En `NOTES.md`, explica qué son el *session leader*, el *controlling terminal* y el *foreground process group*, y qué ocurre cuando un proceso en background intenta leer de la terminal.

CAPÍTULO IV

Parte bonus

- `mpopen` / `mpclose`: tu propia implementación de `popen(3)` y `pclose(3)`. ¿Por qué `pclose` tiene que saber qué PID corresponde a cada `FILE *`?
- `run` con `posix_spawn(3)` en lugar de `fork` + `execve`. Mide la diferencia lanzando 10 000 procesos desde un proceso que tenga 1 GiB de memoria reservada y tocada. Explica el resultado (copy-on-write, `vfork`, `clone`).
- `sigtour` reescrito con `signalfd(2)`. Compara ambos enfoques.

CAPÍTULO V

Evaluación

Prueba	Esperado
`./family 5` + `ps -o pid,stat,cmd`	Sin zombis al final; orden de salida correcto.
`./run ./script_sin_permiso_x`	Código 126 y mensaje claro.
`./run sleep 10` + `kill -9` al hijo	«killed by signal 9 (Killed)», código 137.
`./pipeline "yes" "head -n 1"`	Termina al instante.
`./pipeline "cat /dev/urandom" "head -c 10000000" "wc -c"`	`10000000`.
`./pipeline` con 300 `cat`	Funciona; sin fugas de fds.
`sigtour` + ráfagas `kill -USR1` en bucle	Ni cuelgues ni contadores corruptos; ningún zombi tras un minuto.
`./pgdemo cat`, Ctrl+Z, Enter, Ctrl+D	Se detiene, se reanuda y termina; la terminal sigue usable.
Preguntas sobre los `NOTES.md`	El evaluado las explica sin leer.